



Data Structures

Lecture 11: Graph Traversal

Prof. Wang Haiyan
School of Computer Science
wanghy@njupt.edu.cn

Today's Topic

- Exploring graphs: graph traversal
- Breadth- and Depth-first search
- Shortest paths and breadth-first search

Reminder: Flavours of Graphs

Q: What is probably the biggest man-made graph ever created?

Reminder: Flavours of Graphs

Q: What is probably the biggest man-made graph ever created?

A: The web!

Q: In the web graph, what are the nodes?

Reminder: Flavours of Graphs

Q: What is probably the biggest man-made graph ever created?

A: The web!

Q: In the web graph, what are the nodes?

A: Web pages

Q: What are the edges?

Reminder: Flavours of Graphs

Q: What is probably the biggest man-made graph ever created?

A: The web!

Q: In the web graph, what are the nodes?

A: Web pages

Q: What are the edges?

A: Links

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

... Simple or non-simple?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

... Simple or non-simple?

... Sparse or dense?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

... Simple or non-simple?

... Sparse or dense?

... Cyclic or acyclic?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

... Simple or non-simple?

... Sparse or dense?

... Cyclic or acyclic?

... Explicit or implicit?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

... Simple or non-simple?

... Sparse or dense?

... Cyclic or acyclic?

... Explicit or implicit?

... Embedded or topological?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

... Simple or non-simple?

... Sparse or dense?

... Cyclic or acyclic?

... Explicit or implicit?

... Embedded or topological?

... Labeled or unlabeled?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

... Simple or non-simple?

... Sparse or dense?

... Cyclic or acyclic?

... Explicit or implicit?

... Embedded or topological?

... Labeled or unlabeled?

Q: What data structure is used to store the web graph?

Reminder: Flavours of Graphs

Q: Is the web graph ...

... Directed or undirected?

... Weighted or unweighted?

... Simple or non-simple?

... Sparse or dense?

... Cyclic or acyclic?

... Explicit or implicit?

... Embedded or topological?

... Labeled or unlabeled?

Q: What data structure is used to store the web graph?

A: Most like an adjacency list, each page contains all its outgoing edges

Traversing a Graph

A fundamental graph operation is to visit every edge and node in the graph in a systematic way

This is ***graph traversal***

Think of it as “exploring” a graph in the same way as a robot exploring a maze to escape

We must traverse the graph in a systematic way to ensure all of it is visited

Traversal turns a graph into a ***spanning tree***: we start from a source node (the root), nodes in the graph at distance n from the source become tree nodes at depth n

Q: (Revision) What is the difference between a graph and a tree?

Traversing a Graph

A fundamental graph operation is to visit every edge and node in the graph in a systematic way

This is ***graph traversal***

Think of it as “exploring” a graph in the same way as a robot exploring a maze to escape

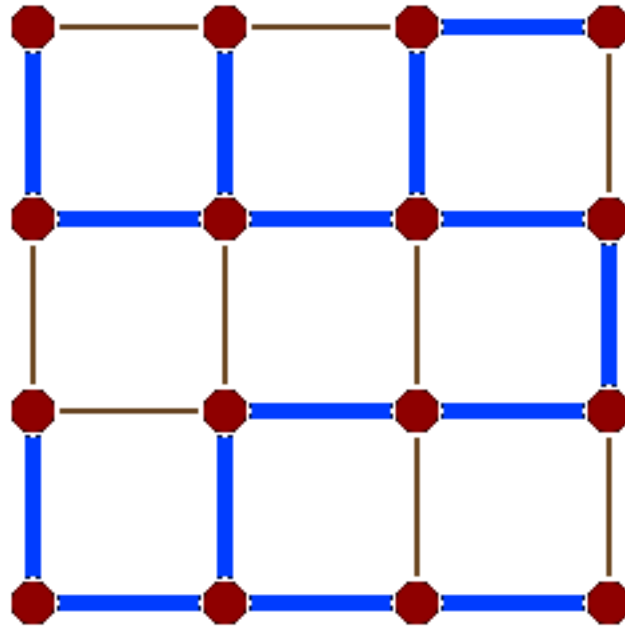
We must traverse the graph in a systematic way to ensure all of it is visited

Traversal turns a graph into a ***spanning tree***: we start from a source node (the root), nodes in the graph at distance n from the source become tree nodes at depth n

Q: (Revision) What is the difference between a graph and a tree?

A: In a tree, all nodes are connected by exactly one path

Spanning Trees



One of the spanning trees of the graph

Traversing a Graph

As we move through a graph, we need to keep track of which nodes we have visited

- For efficiency we don't want to keep visiting the same node
- For correctness we need to be sure all nodes have been visited

We assign states to nodes:

- **Undiscovered**: node has not yet been visited
- **Discovered**: node has been visited, but we have not yet explored all of its outgoing edges
- **Processed**: node visited and all outgoing edges have been explored

In Cormen, these states are visualised as colouring the nodes

Traversing a Graph

We have 2 options when traversing a graph:

1. Be cautious: when we arrive at a node, explore all of its children before exploring any further
2. Be adventurous: when we arrive at a node, pick one of its children and explore this before looking at the other children

Breadth-First Search versus ***Depth-First Search***

Both maintain a “to do” list of the nodes that have been discovered but not yet explored

The only difference between the two is the abstract data structure we use to store discovered nodes

Initially, only the source node is considered discovered

Processing applied to nodes as appropriate – if we’re searching for a node, we can quit once we’ve found it

Graph Traversal Sketches

Breadth-First Search

Add the source node to a queue

While the queue is not empty

 Current node = DeQueue front of queue

 If current node is unvisited

 Process current node as desired

 EnQueue all nodes adjacent to current node

Depth-First Search

Add the source node to a stack

While the stack is not empty

 Current node = Pop top of stack

 If current node is unvisited

 Process current node as desired

 Push all nodes adjacent to current node

Traversing a Graph

- Breadth-First Search will generally result in shallow, wide trees
- Depth-First Search will generally result in deep, thin trees
- Both will eventually visit all nodes
- Both are nondeterministic – we do not prescribe which outgoing edges we should follow first (this will be determined by the data structure used to implement the graph)
- For some applications it might not matter which traversal we use – it will just change the order in which we process the nodes
- For other applications, it may completely change the behaviour of the algorithm if we change the traversal strategy

Breadth-First Search

Function $BFS(G : Graph; s : Node)$

1: **for each** vertex $u \in G.V \setminus \{s\}$ **do**

2: $state[u] = UNDISCOVERED$; $parent[u] = NULL$

3: **end for**

4: $state[s] = DISCOVERED$; $parent[s] = NULL$

5: $Q := \{s\}$

6: **while** $Q \neq \emptyset$ **do**

7: $u := DeQueue(Q)$

8: process vertex u as desired

9: **for each** vertex $v \in Adj[u]$ **do**

10: **if** $state[v] = UNDISCOVERED$ **then**

11: $state[v] = DISCOVERED$

12: $parent[v] = u$

13: $EnQueue(Q, v)$

14: **end if**

15: **end for**

16: $state[v] = PROCESSED$

17: **end while**

Breadth-First Search

Function $BFS(G : Graph; s : Node)$

1: **for each** vertex $u \in G.V \setminus \{s\}$ **do**

2: $state[u] = UNDISCOVERED; parent[u] = NULL$

3: **end for**

4: $state[s] = DISCOVERED; parent[s] = NULL$

5: $Q := \{s\}$

6: **while** $Q \neq \emptyset$ **do**

7: $u := DeQueue(Q)$

8: process vertex u as desired

9: **for each** vertex $v \in Adj[u]$ **do**

10: **if** $state[v] = UNDISCOVERED$ **then**

11: $state[v] = DISCOVERED$

12: $parent[v] = u$

13: $EnQueue(Q, v)$

14: **end if**

15: **end for**

16: $state[v] = PROCESSED$

17: **end while**

} Initialisation

Breadth-First Search

Function $BFS(G : Graph; s : Node)$

```
1: for each vertex  $u \in G.V \setminus \{s\}$  do
2:    $state[u] = UNDISCOVERED; parent[u] = NULL$ 
3: end for
4:  $state[s] = DISCOVERED; parent[s] = NULL$ 
5:  $Q := \{s\}$ 
6: while  $Q \neq \emptyset$  do
7:    $u := DeQueue(Q)$ 
8:   process vertex  $u$  as desired
9:   for each vertex  $v \in Adj[u]$  do
10:    if  $state[v] = UNDISCOVERED$  then
11:       $state[v] = DISCOVERED$ 
12:       $parent[v] = u$ 
13:       $EnQueue(Q, v)$ 
14:    end if
15:  end for
16:   $state[u] = PROCESSED$ 
17: end while
```

Initialisation

Loop until queue is empty

Breadth-First Search

Function $BFS(G : Graph; s : Node)$

```
1: for each vertex  $u \in G.V \setminus \{s\}$  do  
2:    $state[u] = UNDISCOVERED; parent[u] = NULL$   
3: end for  
4:  $state[s] = DISCOVERED; parent[s] = NULL$   
5:  $Q := \{s\}$ 
```

} Initialisation

```
6: while  $Q \neq \emptyset$  do
```

```
7:    $u := DeQueue(Q)$ 
```

```
8:   process vertex  $u$  as desired
```

```
9:   for each vertex  $v \in Adj[u]$  do
```

```
10:    if  $state[v] = UNDISCOVERED$  then
```

```
11:       $state[v] = DISCOVERED$ 
```

```
12:       $parent[v] = u$ 
```

```
13:       $EnQueue(Q, v)$ 
```

```
14:    end if
```

```
15:  end for
```

```
16:   $state[v] = PROCESSED$ 
```

```
17: end while
```

} Loop
through all
the outgoing
edges from
current node

} Loop until
queue is
empty

Breadth-First Search

Function $BFS(G : Graph; s : Node)$

```
1: for each vertex  $u \in G.V \setminus \{s\}$  do
2:    $state[u] = UNDISCOVERED; parent[u] = NULL$ 
3: end for
4:  $state[s] = DISCOVERED; parent[s] = NULL$ 
5:  $Q := \{s\}$ 
6: while  $Q \neq \emptyset$  do
7:    $u := DeQueue(Q)$ 
8:   process vertex  $u$  as desired
9:   for each vertex  $v \in Adj[u]$  do
10:    if  $state[v] = UNDISCOVERED$  then
11:       $state[v] = DISCOVERED$ 
12:       $parent[v] = u$ 
13:       $EnQueue(Q, v)$ 
14:    end if
15:  end for
16:   $state[u] = PROCESSED$ 
17: end while
```

Life-cycle of a node is:

1. Undiscovered
2. Discovered
3. Processed



Breadth-First Search Complexity

Q: Complexity of Breadth-first search?

Breadth-First Search Complexity

Q: Complexity of Breadth-first search?

A: $O(|V| + |E|)$

Every node is enqueued and dequeued once, each operation taking $O(1)$ time

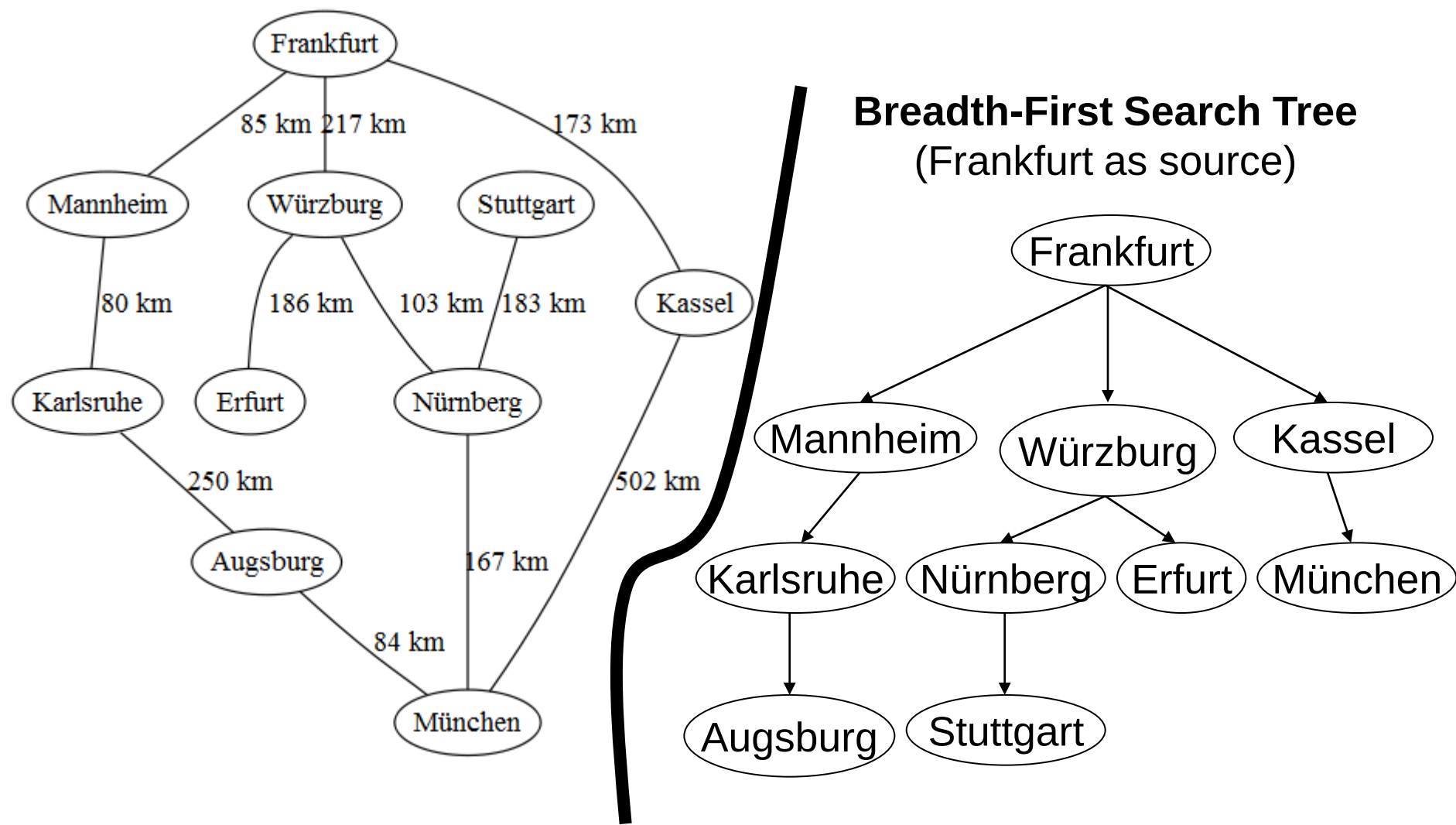
So total time devoted to queueing is $O(|V|)$

Each adjacency list is scanned at most once (when each node is dequeued)

The sum of the lengths of all of the adjacency lists is $O(|E|)$, so total time spent scanning adjacency lists is $O(|E|)$

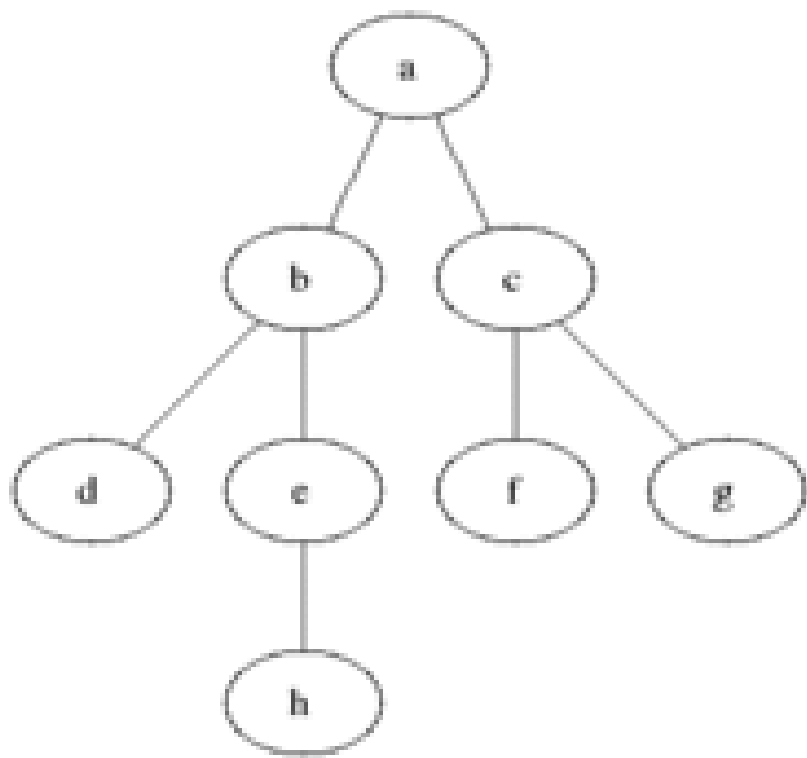
Result is that BFS runs in linear time in the size of the adjacency list representation of the graph

Breadth-First Search Example



Breadth-First Search Animation

Undiscovered = white
Discovered = grey
Processed = black



Paths in Breadth-First Search Trees

- We can find the parent of a node i by looking at $parent[i]$
- The path from a node to the root is defined by the parent relation
- This path satisfies an important property: it uses the smallest possible number of edges from root-to-node
- Conclusion: breadth-first search gives us a way to find shortest paths, but only when the graph is unweighted (i.e. shortest = smallest number of edges)
- Can only be used for shortest paths involving the root and another node
- For paths between two non-root nodes, rebuild the tree

Depth-First Search

Depth-first search has a very simple recursive implementation

Let the compiler manage the stack

Function $DFS(G : Graph; u : Node)$

```
1: state[u] = DISCOVERED
2: process vertex u as desired
3: for each vertex  $v \in Adj[u]$  do
4:   if state[v] = UNDISCOVERED then
5:     parent[v] = u
6:      $DFS(G, v)$ 
7:   end if
8: end for
9: state[u] = PROCESSED
```

Called initially with: $DFS(G, source)$

Depth-First Search

Depth-first search has a very simple recursive implementation

Let the compiler manage the stack

Function $DFS(G : Graph; u : Node)$

```
1: state[u] = DISCOVERED
2: process vertex u as desired
3: for each vertex  $v \in Adj[u]$  do
4:   if state[v] = UNDISCOVERED then
5:     parent[v] = u
6:      $DFS(G, v)$ 
7:   end if
8: end for
9: state[u] = PROCESSED
```

Loop
through all
the outgoing
edges from
current node

Called initially with: $DFS(G, source)$

Depth-First Search

Depth-first search has a very simple recursive implementation

Let the compiler manage the stack

Function $DFS(G : Graph; u : Node)$

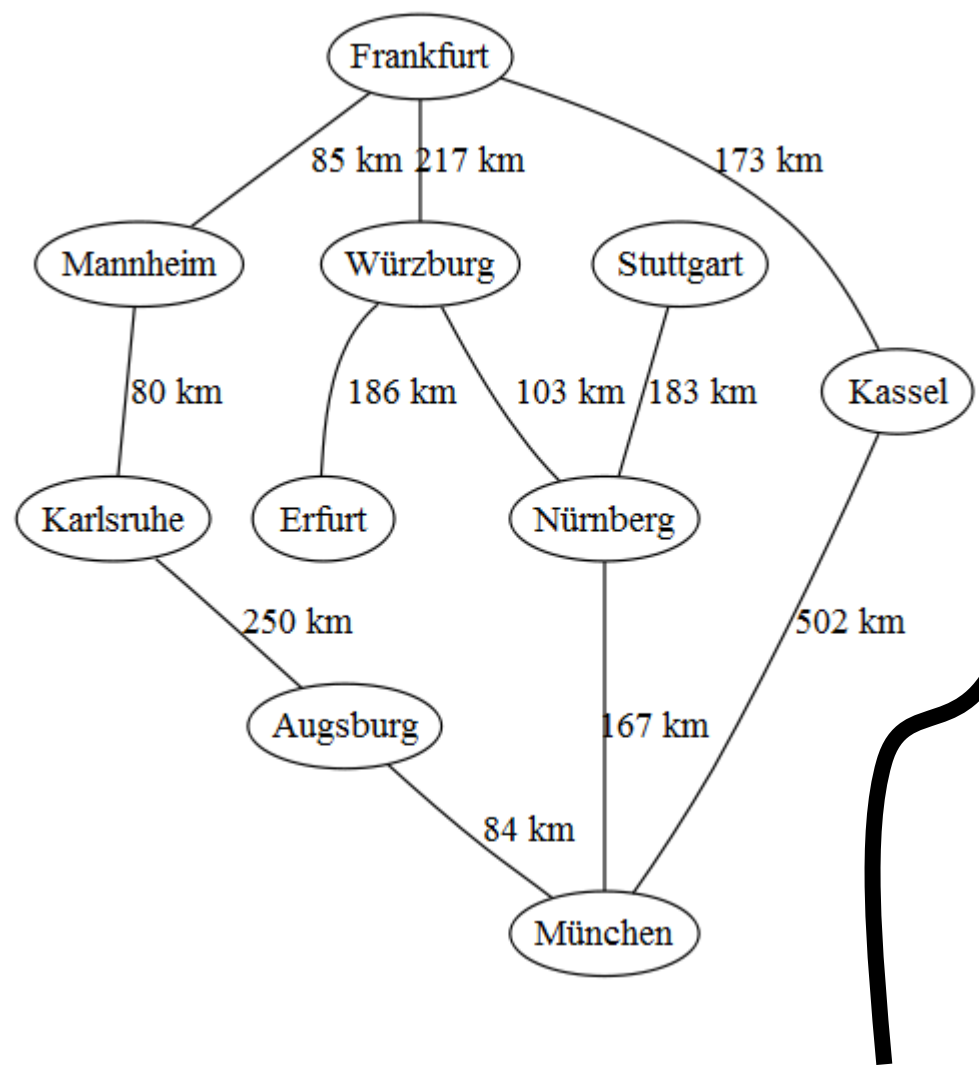
```
1: state[u] = DISCOVERED
2: process vertex  $u$  as desired
3: for each vertex  $v \in Adj[u]$  do
4:   if state[v] = UNDISCOVERED then
5:     parent[v] =  $u$ 
6:      $DFS(G, v)$ 
7:   end if
8: end for
9: state[u] = PROCESSED
```

Recursively call DFS for all outgoing edges to undiscovered nodes

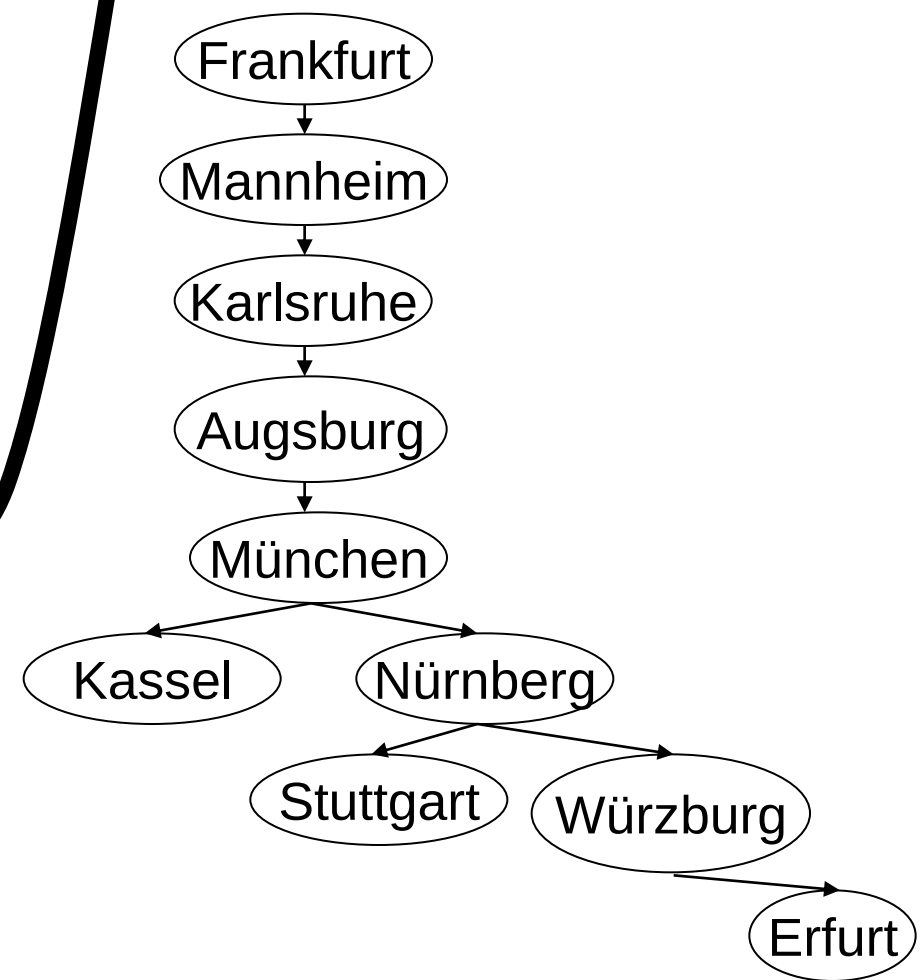
Loop through all the outgoing edges from current node

Called initially with: $DFS(G, source)$

Depth-First Search Example



Depth-First Search Tree
(Frankfurt as source)



Directed Graphs

Examples so far used undirected graphs

Q: Will our pseudocode always visit every node in an undirected graph?

Directed Graphs

Examples so far used undirected graphs

Q: Will our pseudocode always visit every node in an undirected graph?

A: Yes, if the graph is connected – since every node must be connected to another by at least one edge and both algorithms eventually explore all edges

Directed Graphs

Examples so far used undirected graphs

Q: Will our pseudocode always visit every node in an undirected graph?

A: Yes, if the graph is connected – since every node must be connected to another by at least one edge and both algorithms eventually explore all edges

Q: Will our pseudocode still work with directed graphs?

Directed Graphs

Examples so far used undirected graphs

Q: Will our pseudocode always visit every node in an undirected graph?

A: Yes, if the graph is connected – since every node must be connected to another by at least one edge and both algorithms eventually explore all edges

Q: Will our pseudocode still work with directed graphs?

A: Yes, an undirected graph just contains the same edge twice, once for each direction – so our traversal can follow the edge in either direction, in a directed graph the edge may only occur once

Directed Graphs

Examples so far used undirected graphs

Q: Will our pseudocode always visit every node in an undirected graph?

A: Yes, if the graph is connected – since every node must be connected to another by at least one edge and both algorithms eventually explore all edges

Q: Will our pseudocode still work with directed graphs?

A: Yes, an undirected graph just contains the same edge twice, once for each direction – so our traversal can follow the edge in either direction, in a directed graph the edge may only occur once

Q: Will our pseudocode always visit every node in a directed graph?

Directed Graphs

Examples so far used undirected graphs

Q: Will our pseudocode always visit every node in an undirected graph?

A: Yes, if the graph is connected – since every node must be connected to another by at least one edge and both algorithms eventually explore all edges

Q: Will our pseudocode still work with directed graphs?

A: Yes, an undirected graph just contains the same edge twice, once for each direction – so our traversal can follow the edge in either direction, in a directed graph the edge may only occur once

Q: Will our pseudocode always visit every node in a directed graph?

A: Not necessarily: depending on where we start it may not be possible to visit all parts of the graph

Directed Graphs

A directed graph is ***strongly connected*** if there is a path from each vertex in the graph to every other vertex

BFS and DFS will find a spanning tree if the graph is strongly connected

Q: What could we add to our pseudocode to handle directed graphs that are not strongly connected?

Directed Graphs

A directed graph is ***strongly connected*** if there is a path from each vertex in the graph to every other vertex

BFS and DFS will find a spanning tree if the graph is strongly connected

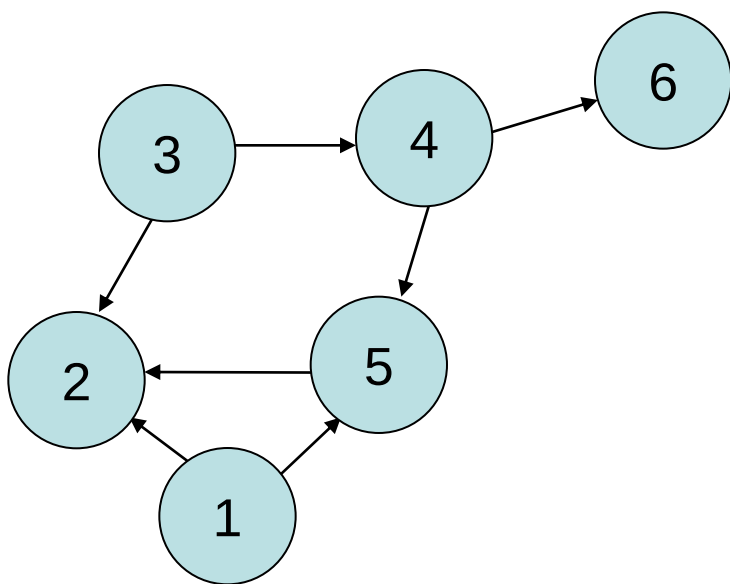
Q: What could we add to our pseudocode to handle directed graphs that are not strongly connected?

A: If our search terminates and there are still undiscovered nodes, we can run the search again with one of these undiscovered nodes as the source, repeating until no nodes are undiscovered

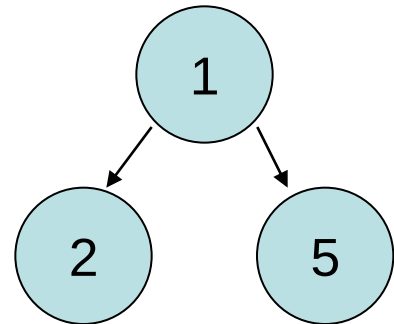
This results in a ***spanning forest***

Directed Graphs

Breadth-first search tree
node 1 as source:

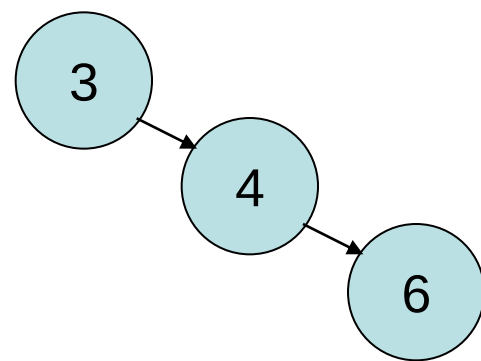


Tree 1



3, 4 and 6 remain unvisited
Run BFS with 3 as source:

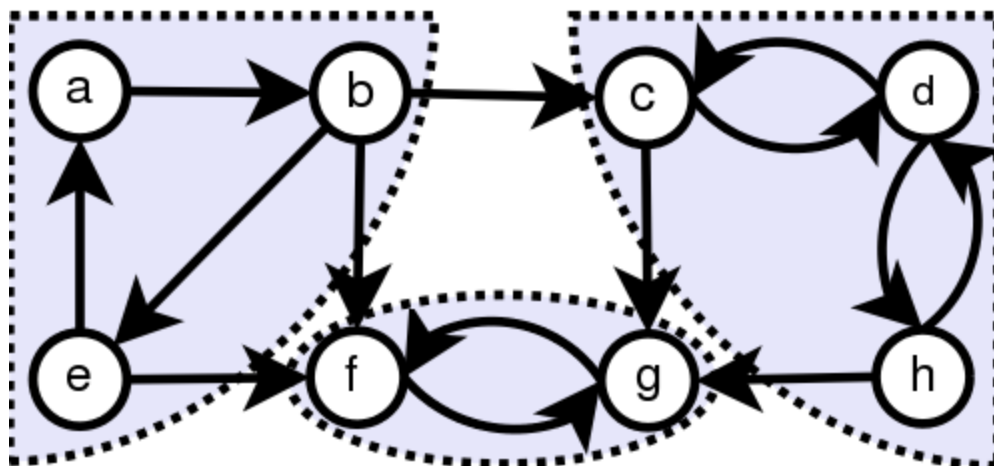
Tree 2



No nodes remain unvisited: the two trees
are a spanning forest of the directed graph

Strongly Connected Components

The ***strongly connected components*** of a directed graph are its maximal strongly connected subgraphs:



If each SCC is contracted to form a single node, the resulting graph is a DAG

SCCs can be found using a number of algorithms which all exploit depth-first search

Conclusions

We should now know:

- The aims and requirements of graph traversal
- Breadth- versus Depth-first-search
- Shortest paths in BFS trees

Next time:

- Weighted graphs: minimum spanning trees

Recommended reading for this lecture:

- Mehlhorn Chapter 9
- Cormen Chapter 22
- Skiena 5.3 to end of Chapter 5